

Entregables

No olvide duplicar esta notebook para poder editar: File->Save a copy in Drive

En este laboratorio, no es necesario redactar un informe en un documento separado. Por favor, complete todas las actividades requeridas dentro de esta notebook de Google Colab. Recuerde que una notebook le permite ingresar elementos de texto de manera similar a un procesador de documentos. Una vez finalizadas las actividades propuestas, deberá entregar lo siguiente en la plataforma:

1. Un archivo PDF generado en Google Colab desde el menú "Archivo" -> "Imprimir".
2. El enlace público de Google Colab. Para ello, vaya al botón de compartir y cambie la configuración de compartición a "Cualquier persona con el enlace".



Comment



Share



Resolución

Una vez que la nota sea entregada, encontrarás la resolución en este link de notion:

https://www.notion.so/resoluciones-estudiantes-mmia-ff346c62ef4742b2a8c21c50f037b17f?source=copy_link

Recuerda que existen varias formas de solucionar las actividades propuestas en los laboratorios. Toma la resolución entregada como referencia. En caso de que el link de resolución no esté público, solicita acceso en el mismo link de notion.

What is Feature Selection?

In the realm of machine learning, feature selection is the process of identifying

and selecting a subset of relevant features for use in model construction. This process is crucial because the quality and quantity of features feeding into a model significantly impact its performance and interpretability. By choosing the right features, we can improve model accuracy, reduce complexity, and increase computational efficiency.

Imagine you're trying to predict the quality of wine based on various attributes (features) like acidity, sugar level, alcohol content, etc. Not all these attributes might be relevant to determining wine quality. Feature selection helps us identify which attributes (features) are essential and which can be ignored, thus simplifying our model without sacrificing accuracy.

Why is Feature Selection Important?

Feature selection is not just about finding the needle in the haystack. It's about recognizing that not all data points are created equal when it comes to the predictive power they hold. Here are a few reasons why feature selection is vital:

1. **Improves Model Accuracy:** Eliminates noise by removing irrelevant or redundant features, thereby enhancing the model's predictive performance.
2. **Reduces Overfitting:** Fewer irrelevant features reduce the chance of making decisions based on noise, leading to a more generalized model.
3. **Increases Model Efficiency:** Less data means faster training. Models train quicker on fewer features.
4. **Enhances Model Interpretability:** A model with fewer features is easier to understand and explain, making the outcomes more actionable.

Curse of dimensionality

The "curse of dimensionality" refers to a set of problems that arise when analyzing and organizing data in high-dimensional spaces (i.e., spaces with a large number of features or variables). As the number of dimensions increases, the volume of the space increases so fast that the available data become sparse, which presents challenges for statistical and machine learning methods.

Key Problems Caused by the Curse of Dimensionality

1. **Distance Metrics Become Less Informative:** In high dimensions, the

concept of "closeness" or "nearness" becomes meaningless. For example, the distance between all points tends to become similar, which negatively affects algorithms like k-NN, clustering, or any method relying on distance.

2. **Data Sparsity:** With each added dimension, the volume of the feature space increases exponentially. This means more data is required to maintain the same density, but in practice, data is often limited. Thus, models trained in high dimensions risk overfitting.
3. **Increased Computational Cost:** Algorithms operating in high-dimensional spaces require more memory and processing time, making them computationally expensive or infeasible.
4. **Model Overfitting:** High-dimensional data allows models to fit noise rather than signal, especially when the number of dimensions exceeds the number of observations.
5. **Difficulty in Visualization and Interpretation:** Human interpretation impossible beyond three dimensions, making exploratory data analysis and intuition-based debugging hard.

Examples

- **Machine Learning:** In a dataset with hundreds or thousands of features (like image or gene data), models might perform worse unless dimensionality is reduced.
- **Search and Retrieval:** In high-dimensional vector spaces (e.g., embeddings in NLP), nearest neighbor searches become inefficient and unreliable.
- **Clustering:** Algorithms like k-means degrade in performance as distances lose meaning.

Common Solutions

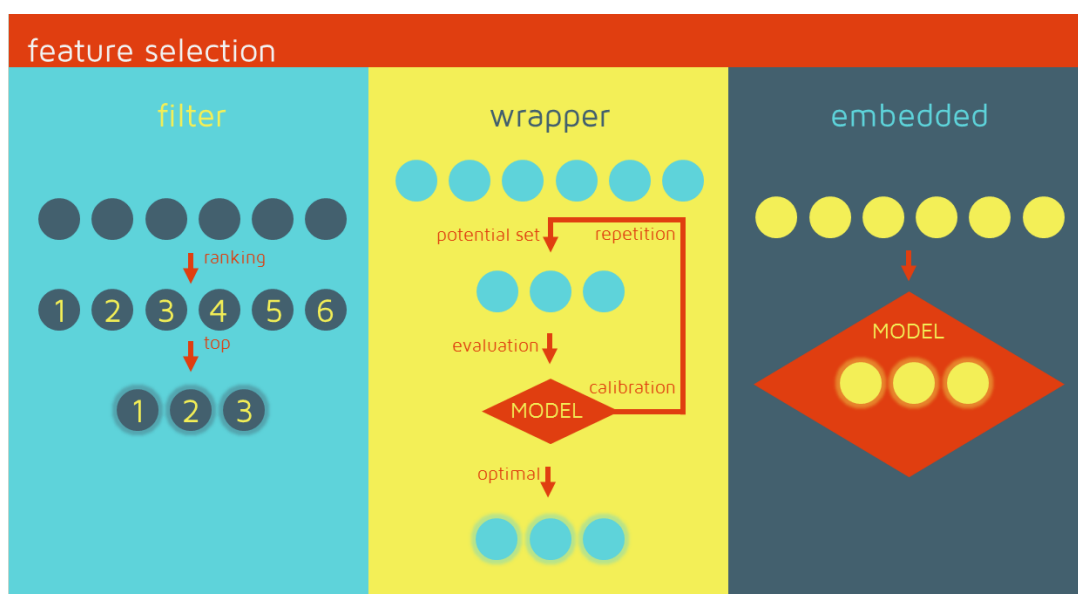
1. **Dimensionality Reduction:**
 - PCA (Principal Component Analysis)
 - t-SNE, UMAP (for visualization)
 - Autoencoders
2. **Feature Selection:** Use only the most relevant features based on statistical

methods or model performance.

3. Regularization: Techniques like L1 (Lasso) or L2 (Ridge) can penalize unnecessary complexity and help reduce the effective number of features.

The Three Main Approaches to Feature Selection

Feature selection techniques are broadly classified into three categories: **Filter Methods**, **Wrapper Methods**, and **Embedded Methods**. Each has its own merits and fits different scenarios.



Filter Methods: Unveiling Patterns through Statistical Scores

Filter methods are a class of feature selection techniques that operate independently of any machine learning algorithm. Instead, they evaluate each feature based on certain statistical measures, and features are selected or discarded before introducing the data to the learning algorithm. This approach offers simplicity, speed, and efficiency in identifying relevant features. Let's explore some key aspects of filter methods:

1. Statistical Scores:

- **Idea:** Filter methods rank features by their statistical scores, which measure the correlation or association between each feature and (in general) the target variable.
- **Score (metrics):** Filter methods employ various statistical metrics, such

as correlation coefficients, chi-square statistics, information gain, or mutual information, depending on the nature of the data (continuous or categorical) and the task at hand.

2. Independence from Algorithms:

- **Advantage:** Filter methods are algorithm-agnostic. They assess features independently of the machine learning model that will eventually be applied. This makes them computationally less expensive and a suitable starting point for feature selection.

3. Feature Ranking:

- **Process:** Features are ranked based on their individual scores. Those with higher scores are deemed more relevant, while lower-ranked features are considered less informative for predicting the target variable.

4. Selection Criteria:

- **Thresholding:** A threshold is set, and features surpassing this threshold are retained, forming the selected subset.

5. Advantages of Filter Methods:

- **Computational Efficiency:** Filter methods are faster compared to wrapper methods and embedded methods, making them suitable for datasets with a large number of features.
- **Algorithm Independence:** Since they don't rely on a specific learning algorithm, filter methods are versatile and can be applied to various types of datasets and models.

6. Disadvantages of Filter Methods:

- **Ignoring Feature Interactions:** Filter methods assess features in isolation and might overlook complex interactions between features. This limitation makes them less suitable for tasks where feature dependencies are crucial.

7. Common Scores where target is required to compute the statistical relationship between each feature and the target:

- **Correlation Coefficient (for regression):** Measures linear relationships between continuous features and the target variable.
- **Chi-Square Test (for classification):** Appropriate for categorical features, it assesses the independence of a feature from the target variable.
- **Mutual Information (for both classification and regression):** Measures the dependence between two variables and is suitable for both

continuous and categorical features.

- **ANOVA F-test (for classification)**

8. **Common Scores where target is not required to compute the statistical relationship:**

- **Variance threshold:** Remove features with very low variance.
- **Correlation threshold:** Remove highly correlated features to avoid redundancy (feature-to-feature correlation).

Wrapper Methods: Iterative Search for Optimal Feature Sets

Wrapper methods for feature selection treat the process as a search problem, involving the evaluation of different subsets of features using a predictive model. Unlike filter methods, wrapper methods actively involve a machine learning algorithm in the feature selection process. This iterative approach seeks to find the optimal set of features by training and evaluating the model multiple times. Here are the key aspects of wrapper methods:

1. **Search Strategies:**

- **Idea:** Wrapper methods treat feature selection as a search problem, exploring different combinations of features to identify the subset that maximizes the performance of a given model.
- **Examples:** Common search strategies include forward selection, backward elimination, and recursive feature elimination (RFE).

2. **Model Evaluation:**

- **Criteria:** A predictive model is used to evaluate each subset of features, and a performance metric (e.g., accuracy, F1 score) is assigned to each combination.
- **Scoring:** The quality of a feature subset is determined by the model's predictive performance on a validation set.

3. **Iterative Process:**

- **Cycle:** Wrapper methods follow an iterative cycle of feature subset evaluation, model training, and refinement.
- **Optimization:** The process continues until an optimal feature subset is found or until a specified stopping criterion is met.

4. **Feature Ranking:**

- **Dynamic:** Features are dynamically selected or excluded in each iteration,

allowing the algorithm to explore the entire feature space.

5. Advantages of Wrapper Methods:

- **Model Sensitivity:** Wrapper methods consider the specific model's sensitivity to feature subsets, making them more suitable for complex relationships and interactions between features.

6. Disadvantages of Wrapper Methods:

- **Computational Cost:** Wrapper methods are computationally more expensive than filter methods, as they require training the model multiple times.
- **Overfitting Risk:** There's a risk of overfitting to the training data, especially if the dataset is small, due to multiple evaluations.

7. Common Wrapper Methods:

- **Forward Selection:** Features are added one at a time, and at each step, the feature that provides the most improvement is included.
- **Backward Elimination:** Starts with all features and removes the least significant one in each iteration until the specified stopping criterion is met.
- **Recursive Feature Elimination (RFE):** An iterative process where the least important features are pruned until the desired number of features is reached.

Embedded Methods: Inherent Feature Selection within Model Training

Embedded methods seamlessly incorporate feature selection into the process of model training. Unlike filter and wrapper methods, embedded methods select features as a natural part of the algorithm's learning process. By doing so, they leverage the inherent capacity of certain algorithms to identify the most informative features. Here are the key aspects of embedded methods:

1. Integration with Model Training:

- **Inherent Selection:** Features are selected or assigned weights during the model training process itself.
- **Learning Process:** The algorithm optimizes both the model parameters and the importance of each feature simultaneously.

2. Advantages of Embedded Methods:

- **Inherent Feature Interactions:** Embedded methods naturally consider

interactions between features as part of the learning process, making them effective for capturing complex relationships.

- **Model-Specific Selection:** Selection is tailored to the characteristics of the chosen model, ensuring a well-fitted subset of features.

3. Disadvantages of Embedded Methods:

- **Computational Cost:** Some embedded methods, especially those involving iterative optimization, can be computationally expensive.
- **Algorithm Dependency:** The effectiveness of embedded methods depends on the suitability of the chosen model for the specific dataset.

6. Common Embedded Methods:

- **LASSO (Least Absolute Shrinkage and Selection Operator):** Regularizes the linear regression model by adding a penalty term based on the absolute sum of feature coefficients.
- **Tree-Based Models (Random Forest, Gradient Boosting):** Assigns importance scores to each feature during the ensemble learning process.

Feature selection with scikit-learn for the Breast Cancer Dataset

The Breast Cancer dataset from scikit-learn is a widely used dataset for binary classification tasks, particularly in the context of cancer diagnosis. It consists of features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. These features describe various characteristics of cell nuclei present in the images, such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.

Here's more detailed information about the Breast Cancer dataset:

- **Dataset Origin:** The dataset is derived from the Wisconsin Diagnostic Breast Cancer (WDBC) database available at <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- **Number of Instances:** There are 569 instances, each corresponding to a breast cancer biopsy. **Number of Features:** The dataset comprises 30 features, representing measurements extracted from digitized images.
- **Target Variable:** The target variable is binary, indicating whether the tumor is malignant (class 0) or benign (class 1).
- **Use Case:** The primary use case for this dataset is binary classification, where the goal is to predict whether a tumor is malignant or benign based

on the extracted features.

- **Data Availability:** This dataset is included in the scikit-learn library, making it easily accessible for educational and research purposes.
- **Research Significance:** Due to its real-world medical context and the critical nature of breast cancer diagnosis, this dataset is frequently employed for exploring machine learning techniques for cancer classification. It provides a challenging yet realistic task for building predictive models.

In the context of your feature selection notebook, using the Breast Cancer dataset serves as a meaningful and relevant example. The task of predicting cancer malignancy based on various features underscores the importance of feature selection in enhancing the interpretability and performance of machine learning models for critical applications in healthcare and diagnostics.

Baseline model using all available variables

In our exploration of feature selection, we begin by loading and preparing the Breast Cancer dataset, a well-known benchmark in machine learning. This dataset encompasses various features related to breast cancer characteristics. To ensure robust model evaluation, we strategically split the dataset into training and testing sets. Subsequently, we standardize the features to bring them to a common scale, a crucial preprocessing step that aids in the fair comparison of their contributions to the model. With our data prepared, we proceed to implement and assess a baseline model, utilizing all available features. This initial model serves as our reference point, providing insights into the dataset's inherent characteristics and paving the way for the subsequent application of feature selection techniques to enhance model performance and interpretability.

```
In [1]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
#from sklearn.feature_selection import SelectKBest, chi2, RFE, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, recall_score,

# Load the dataset
data = load_breast_cancer()
X = data.data
y = data.target
```

```

feature_names = data.feature_names

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define a function to evaluate the model
def evaluate_model(X_train, X_test, y_train, y_test):
    model = LogisticRegression(max_iter=10000)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    score = recall_score(y_test, y_pred)
    return score

print("Baseline Model Performance:")
print(evaluate_model(X_train_scaled, X_test_scaled, y_train, y_test))

```

Baseline Model Performance:
0.9814814814814815

Feature selection using a Filter Method

The following code uses the `SelectKBest` function from the `sklearn.feature_selection` library in Python. This function is used to select the best features from a dataset based on a univariate statistical test. In this case, the code is using the `mutual_info_classif` test to select the 10 best features from the `X_train_scaled` and `X_test_scaled` datasets. The selected features are then stored in the `X_train_k_best` and `X_test_k_best` variables, respectively. The `mutual_info_classif` test measures the mutual information between each feature and the target variable. Mutual information is a measure of how much information one variable contains about another variable. In this case, the mutual information test is used to select the features that are most likely to be relevant to the target variable. The `SelectKBest` function can be used to improve the performance of a machine learning model by reducing the number of features in the dataset. This can make the model more efficient and less likely to overfit the data.

```

In [2]: ### Filter Method ###
# Imports
from sklearn.feature_selection import SelectKBest, mutual_info_classif

print("\n### Filter Method ###")
# Select the best features based on univariate statistical tests

# Create a SelectKBest object using the mutual_info_classif test and
select_k_best = SelectKBest(mutual_info_classif, k=17)

```

```

# Fit the SelectKBest object on the training data
X_train_k_best = select_k_best.fit_transform(X_train_scaled, y_train)

# Transform the test data using the fitted SelectKBest object
X_test_k_best = select_k_best.transform(X_test_scaled)

print(len(feature_names[select_k_best.get_support()]), "Selected Features")
print("Model Recall with Selected Features:")
print(evaluate_model(X_train_k_best, X_test_k_best, y_train, y_test))

```

Filter Method

```

17 Selected Features: ['mean radius' 'mean perimeter' 'mean area' 'mean
mean compactness'
'mean concavity' 'mean concave points' 'radius error' 'perimeter error'
'area error' 'concavity error' 'worst radius' 'worst texture'
'worst perimeter' 'worst area' 'worst compactness' 'worst concavity'
'worst concave points']
Model Recall with Selected Features:
0.9907407407407407

```

This section uses matplotlib to create a bar chart that displays the importance of each feature visually.

```

In [3]: # Get feature importance scores
feature_scores = select_k_best.scores_

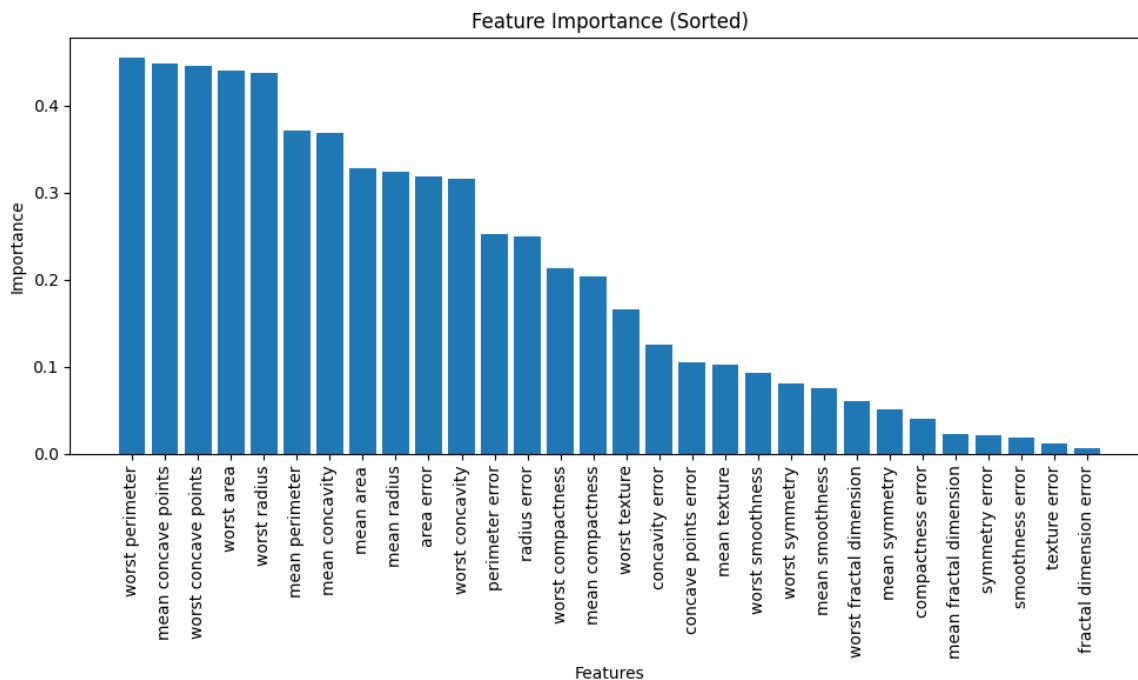
# Sort feature importance scores in descending order
indices = feature_scores.argsort()[::-1]

# Get the names of the sorted features
sorted_feature_names = feature_names[indices]

# Get the sorted feature importance scores
sorted_feature_scores = feature_scores[indices]

# Plot feature importance
plt.figure(figsize=(10, 6))
plt.bar(range(len(feature_names)), sorted_feature_scores)
plt.xticks(range(len(feature_names)), sorted_feature_names, rotation=45)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance (Sorted)")
plt.tight_layout()
plt.show()

```



Feature selection using a Wrapper Method

The following code uses the RFE function from the `sklearn.feature_selection` library in Python. This function is used to select the best features from a dataset using a wrapper method. In this case, the code is using the `LogisticRegression` model as the estimator and selecting the 10 best features from the `X_train_scaled` and `X_test_scaled` datasets. The selected features are then stored in the `X_train_rfe` and `X_test_rfe` variables, respectively. The RFE function works by recursively fitting the model on the dataset and removing the least important features until the desired number of features is reached. The importance of each feature is determined by the model's coefficient.

```
In [4]: ### Wrapper Method ###
# Import the RFE function
from sklearn.feature_selection import RFE

print("\n### Wrapper Method ###")
# Use Recursive Feature Elimination
# Create a LogisticRegression object
model = LogisticRegression(max_iter=10000)

# Create an RFE object using the LogisticRegression model and select
rfe = RFE(estimator=model, n_features_to_select=10)

# Fit the RFE object on the training data
X_train_rfe = rfe.fit_transform(X_train_scaled, y_train)

# Transform the test data using the fitted RFE object
X_test_rfe = rfe.transform(X_test_scaled)

print(len(feature_names[rfe.get_support()]), "Selected Features:",
```

```
print("Model Recall with RFE Selected Features:")
print(evaluate_model(X_train_rfe, X_test_rfe, y_train, y_test))
```

```
### Wrapper Method ###
```

```
10 Selected Features: ['mean concave points' 'radius error' 'area er
ror']
```

```
'fractal dimension error' 'worst radius' 'worst texture'
```

```
'worst perimeter' 'worst area' 'worst concavity' 'worst concave poi
nts']
```

```
Model Recall with RFE Selected Features:
```

```
0.9907407407407407
```

Feature selection with an embedded method

This code demonstrates how to use LASSO regression with cross-validation to select features and subsequently evaluate a model's performance using the selected feature subset. The selected features are based on the non-zero coefficients obtained from the LASSO model.

First, we import the `LassoCV` class from scikit-learn's linear model module. This class is a cross-validated version of LASSO regression. Next, we create a LASSO model (`lasso`) using cross-validation (`cv=5`). The `max_iter` parameter specifies the maximum number of iterations for the optimization algorithm. The model is fitted to the training data (`X_train_scaled` , `y_train`). Finally, we identify the features with non-zero coefficients, indicating that they were selected by the LASSO model. The threshold `1e-05` is used to consider coefficients with absolute values greater than this threshold as non-zero.

```
In [5]: ### Embedded Method ###
from sklearn.linear_model import LassoCV
from sklearn.model_selection import GridSearchCV, KFold
print("\n### Embedded Method ###")

# Fit the Lasso model with cross-validation to find the best alpha
lasso = LogisticRegression(penalty='l1', solver='saga', max_iter=1000)

# Define the grid of hyperparameters to search
param_grid = {'C': np.arange(0.2, 0.3, 0.01)}

# Initialize the GridSearchCV object
grid_search = GridSearchCV(estimator=lasso, param_grid=param_grid, cv=5)

# Fit the grid search object to the data
grid_search.fit(X_train_scaled, y_train)

# Get the best hyperparameters
best_params = grid_search.best_params_

# Get the best model
```

```

best_model = grid_search.best_estimator_

# Get the coefficients (theta)
coef = best_model.coef_

print(coef)
# Identify the non-zero coefficients (selected features)
selected_features = np.where(np.abs(coef)>1e-05)[1]

# Select data for non-zero features
X_train_lasso = X_train_scaled[:, selected_features]
X_test_lasso = X_test_scaled[:, selected_features]

print(len(feature_names[selected_features]), "Selected Features:",
print("Model Recall with LASSO Selected Features:")
print(evaluate_model(X_train_lasso, X_test_lasso, y_train, y_test))

```

Embedded Method

```

[[ 0.          0.          0.          0.          0.          0.
   0.         -1.13522136  0.          0.         -1.06704136  0.
   0.          0.          0.          0.          0.          0.
   0.          0.20526287 -2.45997135 -1.07003956  0.          0.
  -0.24040553  0.         -0.41186542 -0.57925562 -0.33716328  0.]
]
```

9 Selected Features: ['mean concave points' 'radius error' 'fractal dimension error' 'worst radius' 'worst texture' 'worst smoothness' 'worst concavity' 'worst concave points' 'worst symmetry']
Model Recall with LASSO Selected Features:
0.9814814814814815

Hybrid Methods

Hybrid methods that combine elements of both filter and wrapper methods are often employed to harness the benefits of both approaches. These hybrid methods aim to leverage the efficiency of filter methods in quickly preselecting relevant features and the model-awareness of wrapper methods in fine-tuning the feature subsets. Here are a few hybrid methods:

- **Filter-Wrapper Hybrid:**
- **Approach:** Initially, a filter method is used to preselect a subset of features based on some statistical measure (e.g., correlation, information gain). Then, a wrapper method is applied to fine-tune the selected subset using a predictive model.
- **Benefits:** This approach aims to capitalize on the speed of filter methods for an initial feature selection and the model-awareness of wrapper methods to consider feature interactions.
- **Filter-Embedded Hybrid:**
- **Approach:** Filter methods are used to preselect a subset of features based on some criteria. The selected features are then incorporated into

an embedded method during model training. For example, features identified by a filter method may be used in LASSO regression.

- **Benefits:** This hybrid approach combines the efficiency of filter methods with the model-awareness of embedded methods, allowing for simultaneous feature selection and model training.
- **Wrapper-Filter Hybrid:**
- **Approach:** Similar to the filter-wrapper hybrid, a wrapper method is used to iteratively search for the best subset of features. Within each iteration, a filter method is applied to quickly evaluate feature subsets.
- **Benefits:** This hybrid method aims to strike a balance between the exhaustive search of wrapper methods and the efficiency of filter methods, potentially reducing the computational cost.
- **Filter-Wrapper-Embedded Hybrid:**
- **Approach:** This comprehensive hybrid method combines all three approaches. It starts with a filter method for initial feature selection, employs a wrapper method for further refinement, and integrates embedded methods during model training.
- **Benefits:** By combining filter, wrapper, and embedded methods, this approach aims to capture the advantages of each, providing a well-rounded feature selection process.

Implementing hybrid methods requires careful consideration of the dataset characteristics, computational resources, and the specific goals of feature selection. The choice of hybrid method can be tailored based on the problem at hand and the trade-offs between computational efficiency and model interpretability.

Activity: Hybrid Method

In the Cancer Database, apply Mutual Information as a filter method to select the top k features. Then, use Recursive Feature Elimination (RFE) with Logistic Regression as the wrapper method on the selected features from Mutual Information. Set `n_features_to_select` to 10 for RFE. Finally, train a Logistic Regression model on the features selected by the filter-wrapper hybrid method and evaluate its performance on the test set.

```
In [8]: ### Hybrid Method: Filter + Wrapper ###

from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score

print("\n### Hybrid Method: Mutual Information + RFE ###")

# 1. Filter: Mutual Information
```

```

k_filter = 17
filter_selector = SelectKBest(
    score_func=lambda X, y: mutual_info_classif(X, y, random_state=
    k=k_filter
)
X_train_filter = filter_selector.fit_transform(X_train_scaled, y_train)
X_test_filter = filter_selector.transform(X_test_scaled)

filter_selected_indices = np.where(filter_selector.get_support())[0]
filter_selected_names = feature_names[filter_selected_indices]

print(f"Features selected by Mutual Information ({k_filter} features):")
print(filter_selected_names)

# 2. Wrapper: RFE sobre las features ya filtradas
model = LogisticRegression(max_iter=10000, random_state=42)
rfe_hybrid = RFE(estimator=model, n_features_to_select=10)

X_train_hybrid = rfe_hybrid.fit_transform(X_train_filter, y_train)
X_test_hybrid = rfe_hybrid.transform(X_test_filter)

hybrid_selected_indices = filter_selected_indices[rfe_hybrid.get_support()]
hybrid_selected_names = feature_names[hybrid_selected_indices]

print(f"\nFeatures selected by the Hybrid Method ({len(hybrid_selected_names)} features):")
print(hybrid_selected_names)

# 3. Evaluación
hybrid_recall = evaluate_model(X_train_hybrid, X_test_hybrid, y_train)
print("\nModel Recall with Hybrid Selected Features:")
print(hybrid_recall)

```

Hybrid Method: Mutual Information + RFE

Features selected by Mutual Information (17 features):

```

['mean radius' 'mean perimeter' 'mean area' 'mean compactness'
 'mean concavity' 'mean concave points' 'radius error' 'perimeter error'
 'area error' 'concavity error' 'worst radius' 'worst texture'
 'worst perimeter' 'worst area' 'worst compactness' 'worst concavity'
 'worst concave points']

```

Features selected by the Hybrid Method (10 features):

```

['mean compactness' 'mean concave points' 'radius error' 'area error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst concavity' 'worst concave points']

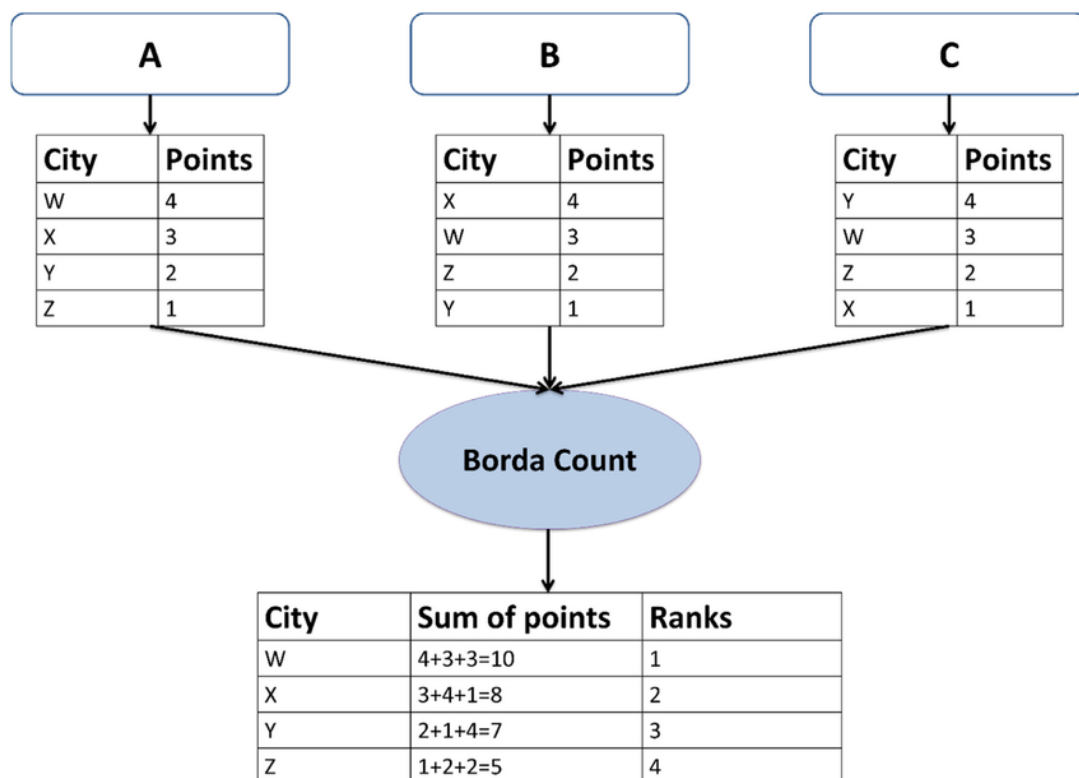
```

Model Recall with Hybrid Selected Features:

```
0.9907407407407407
```



Assignment: Implementing Borda Voting for Feature Selection



Objective: The goal of this assignment is to understand and implement a Borda voting scheme to combine the results of multiple feature selection methods. You will apply this scheme to the Breast Cancer dataset to select a final subset of features and evaluate the performance of a model trained on these selected features.

Understanding Borda Voting:

Imagine you have a group of friends trying to decide on the best movie to watch. Each friend has their own ranked list of preferred movies. How do you combine these individual preferences into a single group ranking? Borda voting is a simple and effective way to do this. It assigns points to each item (in our case, features) based on its position in each individual ranking. The item ranked highest by an individual receives the most points, the second highest receives the next most, and so on. The total Borda score for an item is the sum of the points it received from all individuals. The items with the highest total Borda scores are considered the group's top choices. In the context of feature selection, each individual "friend" is a different feature selection method (Filter, Wrapper, Embedded). Each method provides a ranking of the features. Borda voting allows us to combine these rankings to get a consensus ranking of features.

Your Task:

You will implement a Borda voting scheme to aggregate the feature rankings from four different feature selection methods applied to the Breast Cancer dataset:

1. **Filter Method:** Mutual Information (SelectKBest with mutual_info_classif).
2. **Wrapper Method:** Recursive Feature Elimination (RFE) with LogisticRegression.
3. **Embedded Method:** LASSO Regression (LassoCV).
4. **Embedded Method:** Random Forest Feature Importance (RandomForestClassifier).

Here's a breakdown of the steps you need to follow in your code:

Step 1: Apply Multiple Feature Selection Methods

- For each of the four specified feature selection methods, apply it to the scaled training data (X_train_scaled, y_train).
- The crucial output from each method for Borda voting is a ranking of the features. You need to extract or derive this ranking.
 - For SelectKBest, you can use the scores_ attribute and then np.argsort to get the rank.
 - For RFE, the ranking_ attribute gives the rank directly (be mindful that RFE ranks from 1, so you might need to adjust).
 - For LassoCV, the magnitude of the coef_ indicates importance. Use np.abs() and np.argsort to get the rank.
 - For RandomForestClassifier, the feature_importances_ attribute gives importance. Use np.argsort to get the rank.

Step 2: Implement the Borda Voting Scheme

- Initialize an array (e.g., borda_scores) to store the Borda points for each feature. This array should have a length equal to the total number of features.
- Define a function (or implement the logic directly) to calculate the Borda points for a given ranking. A simple way is to assign points as n_features - rank, where rank is the position in the sorted list (0 for the first, 1 for the second, etc.).
- Iterate through the rankings obtained in Step 1. For each ranking, calculate the Borda points for each feature using your point assignment scheme and add these points to the borda_scores array.

Step 3: Aggregate Ranks and Select Top Features

- Once you have the total borda_scores for all features, rank the features based on these scores in descending order. Features with higher total

Borda scores are considered more important by consensus.

- Decide on the number of top features (`n_selected_features`) you want to select.
- Identify the indices of the features that fall within the top `n_selected_features` based on the aggregated Borda ranking.
- Use these indices to retrieve the names of the selected features from the original `feature_names`.

Step 4: Train and Evaluate Model

- Create new training and testing datasets that contain only the features selected in Step 3.
- Train a LogisticRegression model on this reduced training dataset.
- Evaluate the performance of the trained model on the reduced testing dataset using relevant metrics (e.g., accuracy, recall, F1 score, precision, ROC AUC score).

Reflection Questions:

Please answer the following questions to reflect on this activity:

1. Compare the set of features selected by the Borda voting method with the features selected by the individual methods you implemented earlier (Filter, Wrapper, Hybrid). Are there any features that consistently appear in the top selections across multiple methods?
2. How does the performance of the model trained on the Borda-selected features compare to the baseline model (using all features) and the models trained on features selected by individual methods? What might explain any differences observed?
3. What are the potential advantages of using a consensus-based approach like Borda voting for feature selection compared to relying on a single method?
4. Can you think of any potential disadvantages or limitations of using Borda voting for feature selection?

```
In [21]: # Imports (already provided in the notebook)
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression, LassoCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
import numpy as np
```

```

import pandas as pd

print("\n### Borda Voting for Feature Selection ###")

n_features = X_train_scaled.shape[1]

# Helper: evaluate with several metrics (accuracy, recall, F1 score)
def evaluate_model_full(X_train_sel, X_test_sel, y_train, y_test):
    model = LogisticRegression(max_iter=10000, random_state=42)
    model.fit(X_train_sel, y_train)
    y_pred = model.predict(X_test_sel)
    y_proba = model.predict_proba(X_test_sel)[:, 1]
    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "roc_auc": roc_auc_score(y_test, y_proba)
    }

# --- 1. Apply Feature Selection Methods and get rankings ---
# Apply SelectKBest (Mutual Information) and get rankings
filter_selector = SelectKBest(
    score_func=lambda X, y: mutual_info_classif(X, y, random_state=42),
    k="all"
)
filter_selector.fit(X_train_scaled, y_train)
filter_scores = filter_selector.scores_
filter_ranking = np.argsort(filter_scores)[:, :-1]

# Apply RFE (Logistic Regression) and get rankings
rfe_selector_all = RFE(
    estimator=LogisticRegression(max_iter=10000, random_state=42),
    n_features_to_select=1
)
rfe_selector_all.fit(X_train_scaled, y_train)
rfe_ranking = np.argsort(rfe_selector_all.ranking_)

# Apply LassoCV and get rankings
lasso_cv = LassoCV(cv=5, random_state=42, max_iter=10000)
lasso_cv.fit(X_train_scaled, y_train)
lasso_importance = np.abs(lasso_cv.coef_)
lasso_ranking = np.argsort(lasso_importance)[:, :-1]

# Apply RandomForestClassifier and get rankings
rf_model = RandomForestClassifier(n_estimators=500, random_state=42)
rf_model.fit(X_train_scaled, y_train)
rf_importance = rf_model.feature_importances_
rf_ranking = np.argsort(rf_importance)[:, :-1]

# Show the top 10 features from each individual method
method_rankings = {
    "Mutual Information": filter_ranking,
    "RFE Logistic Regression": rfe_ranking,
    "LassoCV": lasso_ranking,
    "Random Forest": rf_ranking
}

```

```

}

print("\nTop 10 features by individual method:")
for method_name, ranking in method_rankings.items():
    print(f"{method_name}:")
    for item in feature_names[ranking[:10]]:
        print(f" - {item}")
    print()

# --- 2. Implement Borda Voting Scheme ---
# Initialize borda_scores array
borda_scores = np.zeros(n_features)

# Define function to assign points based on ranks
def add_borda_points(ranking, scores_array):
    """Assign n_features points to the best feature, n_features-1 to
    the second best, and so on"""
    for position, feature_idx in enumerate(ranking):
        scores_array[feature_idx] += n_features - position
    return scores_array

# Add points from each method's ranking to borda_scores
for ranking in method_rankings.values():
    borda_scores = add_borda_points(ranking, borda_scores)

# --- 3. Aggregate Ranks and Select Top Features ---
# Rank features based on total borda_scores
borda_ranking = np.argsort(borda_scores)[::-1]

# Define n_selected_features
n_selected_features = 10

# Get indices and names of selected features
borda_selected_indices = borda_ranking[:n_selected_features]
borda_selected_features = feature_names[borda_selected_indices]

borda_results = pd.DataFrame({
    "feature": feature_names[borda_ranking],
    "borda_score": borda_scores[borda_ranking]
})

print(f"\nFeatures selected by Borda Voting ({n_selected_features})")

# Print selected feature names
print(list(borda_selected_features))

print("\nTop Borda ranking:")
print(borda_results.head(n_selected_features).to_string(index=False))

# --- 4. Train and Evaluate Model ---
# Create new training and testing sets with selected features
X_train_borda = X_train_scaled[:, borda_selected_indices]
X_test_borda = X_test_scaled[:, borda_selected_indices]

# Train LogisticRegression model
borda_metrics = evaluate_model_full(X_train_borda, X_test_borda, y_train, y_test)
baseline_metrics = evaluate_model_full(X_train_scaled, X_test_scaled, y_train, y_test)

```

```
# Evaluate model and print metrics
comparison = pd.DataFrame([baseline_metrics, borda_metrics], index=[
    'Baseline', 'Borda Voting'])

# agregar cantidad de features seleccionadas
comparison["n_features"] = [X_train_scaled.shape[1], n_selected_features]

print("\nModel performance comparison:")
print(comparison.to_string())
```

Borda Voting for Feature Selection

Top 10 features by individual method:

Mutual Information:

- worst perimeter
- mean concave points
- worst concave points
- worst area
- worst radius
- mean perimeter
- mean concavity
- mean area
- mean radius
- worst concavity

RFE Logistic Regression:

- worst area
- worst concave points
- worst radius
- radius error
- worst texture
- mean concave points
- area error
- worst concavity
- fractal dimension error
- worst perimeter

LassoCV:

- worst radius
- worst area
- mean concave points
- mean compactness
- radius error
- worst concavity
- concavity error
- area error
- worst texture
- concave points error

Random Forest:

- worst concave points
- mean concave points
- worst area
- worst perimeter
- worst radius
- mean concavity

- worst concavity
- mean perimeter
- mean area
- mean radius

Features selected by Borda Voting (10 features):

```
[np.str_('worst area'), np.str_('mean concave points'), np.str_('worst radius'), np.str_('worst concave points'), np.str_('worst concavity'), np.str_('radius error'), np.str_('area error'), np.str_('worst perimeter'), np.str_('worst texture'), np.str_('mean concavity')]
```

Top Borda ranking:

feature	borda_score
worst area	114.0
mean concave points	111.0
worst radius	110.0
worst concave points	107.0
worst concavity	93.0
radius error	89.0
area error	87.0
worst perimeter	83.0
worst texture	82.0
mean concavity	77.0

Model performance comparison:

oc_auc	n_features	accuracy	recall	f1	precision	r
Baseline	all features	0.982456	0.981481	0.986047	0.990654	0.997942
	30					
Borda selected features	10	0.970760	0.981481	0.976959	0.972477	0.997795

Conclusión

El método híbrido demuestra que es posible reducir significativamente el número de características manteniendo un desempeño prácticamente óptimo. La combinación de Mutual Information y RFE permitió reducir las variables de 30 a 10, conservando únicamente aquellas con mayor capacidad predictiva. A pesar de esta reducción del 67%, el modelo alcanzó un **recall de 0.9907**, evidenciando una excelente capacidad para identificar correctamente los casos positivos.

Por otro lado, Borda Voting logró identificar un subconjunto compacto y robusto de características consensuadas por diferentes métodos de selección. Con solo 10 variables, el modelo obtuvo un **accuracy de 0.9708**, un **F1-score de 0.9770**, un **ROC-AUC de 0.9978** y mantuvo un **recall de 0.9815**, prácticamente igual al modelo base entrenado con las 30 características originales. Esto demuestra que las variables seleccionadas conservan la información crítica para la detección de tumores malignos.

Ambos enfoques lograron una reducción del 67% de las variables manteniendo un rendimiento muy elevado. El método híbrido obtuvo el mejor recall (**0.9907** frente a **0.9815**), mientras que Borda Voting produjo un conjunto de características más robusto y respaldado por múltiples técnicas de selección. Para aplicaciones médicas donde minimizar falsos negativos es prioritario, el método híbrido sería la alternativa más atractiva debido a su recall ligeramente superior, mientras que Borda Voting ofrece una selección más estable e interpretable al integrar el consenso de varios métodos.